Civil Engineering Case Study

- Police (999) Call Processing System

Zichen Jing (Nancy) zj2256
ENGR-UH 1000, Spring 2023 (Assignment 3)

Step 1: Problem Identification and Statement

This assignment aims to design a police call system that helps a call from a citizen connect to the nearest available police when they call 999. The program will simulate a police call processing system to link the call to a police car.

Step 2: Gathering Information and input/Output diagram

- *Gathering of information*

The program will design the system by separating it into three components: police cars, calls, and the police station.
The police cars component will record the police car in the "city". Includes their unique ID, positions in 2D coordinates (x, y coordinates simulation), which police station it belongs to, and their minimum urgent level. The total number of police cars in a police station will be recorded, in this case, there will be only one police station. Thus, the total number of police cars represents the whole city.
The call component will record the information about the call from an accident. Including their unique ID, the position (2D coordinate) of where the accident occurred, and its urgent level.
The police station component stores all the information about police cars and controls their positions, receives the call and assigns suitable police to it. It will then record all the calls as well. The area it covers in this program will be from -100 to 100 for both x and y coordinates.

The urgent level in the "call" and the minimum urgent level in "police cars" is divided into three levels: 1 to be urgent, 2 to be very urgent, and 3 to be extremely urgent. As the associating number increases, the urgent level also increases. The police car can only be assigned to an accident if it has a lower minimum urgent level than the urgent level of the accident. For example, a police car with a minimum urgent level of 1 can be assigned to the call with an accident of urgent level 1, 2 or 3.
In this program, it is assumed that there will only be one police station located at (0,0). All police cars will be assigned to this police station by referencing them to this police station. The information about the police station including its 2D position and ID will be determined internally by the program. The police station will keep track of the number of police cars and the number of calls, initially both values are set to 0. A constructor is created to allow the user to set the position for more police stations, their ID will still be determined internally by the program. However, this constructor will not be used in this assignment.

- *Input/Output Description*

Input:

1. Number of police cars the user wants to add
2. The details about the call (its position including x and y coordinates, its urgent level)
   - Any input outside -100 to 100 will be seen as an error for the accident's position. Input other than 1, 2, or 3 will be seen as an error for the urgent level.

The inputs then become the selection of the number in the menu displayed on the screen

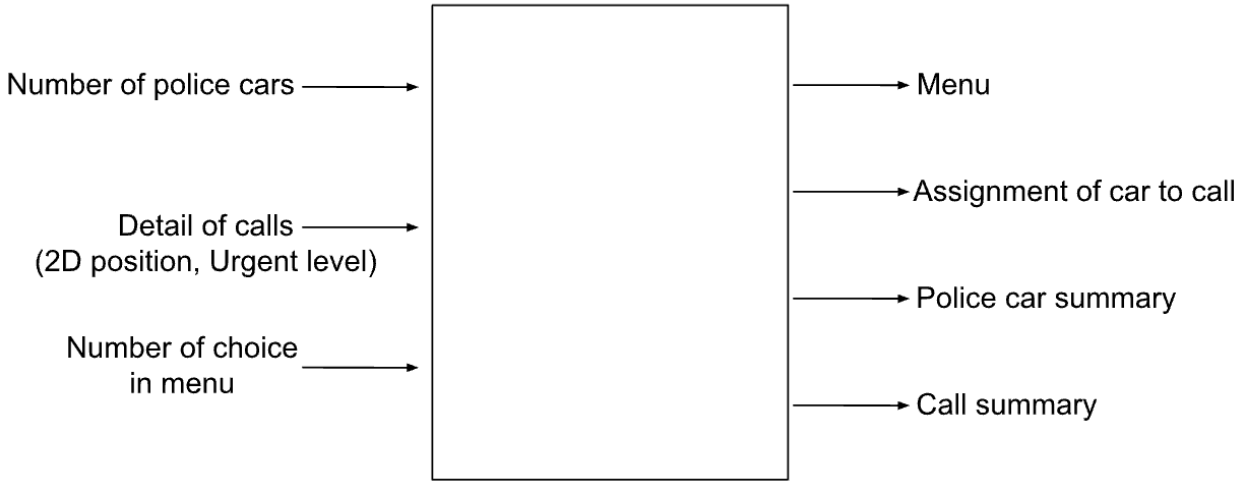
1. Input '1' if the user wants to add more police cars
   - Further input the number of police cars the user wants to add.
2. Input '2' if the user wants to add more calls
   - Further input the details about the car.
3. Input '3' to exit, which terminates the program
4. Any input other than 1, 2, or 3 will be seen as an error

Output:

1. Prompt the user for the number of police cars he/she wants to add
   - Display the information about the added police cars (ID, minimum urgent level, and its position including x and y coordinates)
2. Prompt the user for the detail of the call
   - Display the information about the accident, the assigned police car (most suitable based on having the nearest distance and matching urgent levels), and the distance between them. Information about the remaining (available) police cars will also be displayed. If the user's input about the accident's 2D position is outside -100 to 100 or the urgent level is not 1, 2, or 3. An error message will be displayed. Prompt the user for another value.
3. A menu is displayed as output with three options
   - Corresponding to input 1: Prompt the user for the number of police cars he/she wants to add. Display the information about the added police cars, including the newly added police cars and the remaining police cars since the last call. Police cars that have been assigned to accidents are excluded since they are no longer available to assign. Previous call information will be displayed.
   - Corresponding to input 2: Prompt the user for the detail of the call. Display the information about the accident, the assigned police car (most suitable), and the distance between them. Information about the remaining (available) police cars and the calls including the previous ones will also be displayed. If the user's input about the accident's 2D position is outside -100 to 100 or the urgent level is not 1, 2, or 3. An error message will be displayed. Prompt the user for another value.
   - Corresponding to input 3: Display a goodbye message and then terminates the program.

- Corresponding to any other input: An error message will be displayed. Prompt the user for another value.

Inputs and outputs are also shown in the diagram below:



Step 3: Test cases and Algorithm Design

- *Test Cases*

Test Case 1: Adding police cars

The program starts by asking the user how many police cars the user wants to add to the police station. Based on the user's input, the program will generate accordingly and display the information about the police cars. The information includes the ID, x-coordinate, y-coordinate and its minimum urgent level.
Adding 5 police cars will be tested.

Test Case 2: Add a call (valid inputs)

The program will ask the user about the accident (simulate a call from the accident to 999), including x, and y coordinates, and its urgent level. The program will then assign a police car that is most suitable for this accident (nearest distance and matching urgent level). The program will notify the user about the call (ID, urgent level), the assigned police car (ID, minimum urgent level), and the distance between them. Information about the remaining police cars and the received call will be displayed as two separate tables. The program will continue by displaying a menu for the user to choose from.
Adding a call with position (50, 50) and an urgent level of 2 will be tested.

Test Case 3: Add a call (invalid inputs)

If the user's inputs of the accident are invalid, such as inputs for the 2D position are outside of -100 to 100 or the input for the urgent level is a number outside of 1 to 3. The program will display an error message and prompt the user for another value.

Sample of the main menu

Please choose what you want to do next: 1) Add police car(s) 2) Add a call 3) Exit

Test Case 4: Invalid input in the menu selection

In the menu, users need to select the function they wish to do next. If the user enters an invalid input (numbers other than 1, 2, or 3), the program will display an error message and prompt the user for another valid value.

Test Case 5: Selecting 1) Add police car(s)

If the user selected 1) Add police car(s), the program will display the same content as test case 1. The information about police cars will be updated to include the remaining (unassigned) police cars and the newly added police cars. The information about previous calls will also be displayed. After adding the desired number of police cars, the menu will be redisplayed for the user to choose the next step.
Adding 3 additional police cars will be tested.

Test Case 6: Selecting 2) Add a call

If the user selected 2) Add a call, the program will display the same content as test case 2. The program will still check if the user's inputs are valid, if invalid, the program will display the same content as test case 3. The displayed information about remaining police cars and calls will be updated according to the user's action. After adding the call, the menu will be redisplayed for the user to choose the next step.
Adding a call with position (0, 0) and an urgent level of 1 will be tested.

Test Case 7: Selecting 3) Exit

If the user selected 3) Exit, the program will display a message, such as "Thank you for using the program, goodbye!". And then quit the program.

<u>Note:</u> When there is no suitable police car to assign to an accident, a message will be displayed to notify the user. However, since it is totally random and unpredictable how many tests will be needed for it to occur, it is not included as a test case.

- *Algorithm Design*

Declare class PoliceStation

Declare class PoliceCar

<u>Create private members:</u>
Declare integer: ID, mul
Declare double: xcoordinate, y coordinate
Declare static integer: numOfCars
Declare PoliceStation*: policestation
<u>Create public members:</u>
PoliceCar (pass PoliceStation*: policestation)
    Assign 0 to xcoordinate
    Assign 0 to ycoordinate
    Assign 0 to mul
    Increment numOfCars
    Assign numOfCars to ID
    Assign policestation to policestation in this class
PoliceCar (pass double: xcoordinate, ycoordinate; integer: mul; PoliceStation*: policestation)
    Assign xcoordinate to xcoordinate in this class
    Assign ycoordinate to ycoordinate in this class
    Assign mul to mul in this class
    Increment numOfCars
    Assign numOfCars to ID
    Assign policestation to policestation in this class
int getID
  Return ID
double getPosx
    Return xcoordinate
double getPosy
    Return ycoordinate
int getLevel
  Return mul
int getNumOfCars
  Return numOfCars
void setPosx (pass double: xcoordinate)
  Assign xcoordinate to xcoordinate in this class
void setPosy (pass double: ycoordinate)
  Assign ycoordinate to ycoordinate in this class

void setLevel (pass integer: mul)
    Assign mul to mul (in this class)
void move
    Assign the value returned by function ‘rand_pos’ to xcoordinate
    Assign the value returned by function ‘rand_pos’ to ycoordinate
    Assign the value returned by function ‘rand_mul’ to mul
double rand_pos
        Return a value between -100 and 100 (-100 and 100 included)
double rand_mul
        Return a value between 1 and 3 (1 and 3 included)
void print
    Print (start from the left and set width to 10) ID, (start from the left and set width to 13) mul, (start from the left) “(”, (start from the left) xcoordinate, (start from the left) “,”, (start from the left) ycoordinate, (start from the left and set width to 10) “)”, newline
Assign 0 to the static integer variable numOfCars in class PoliceCar

Declare class Call
<u>Create private members:</u>
Declare integer: CID, ul
Declare double: xcoordinate, y coordinate
Declare static integer: numOfCalls
<u>Create public members:</u>
Call (pass PoliceStation*: policestation)
    Assign 0 to xcoordinate
    Assign 0 to ycoordinate
    Assign 0 to ul
    Increment numOfCalls
    Assign numOfCalls to CID
Call (pass double: xcoordinate, ycoordinate; integer: ul)
    Assign xcoordinate to xcoordinate in this class
    Assign ycoordinate to ycoordinate in this class
    Assign ul to ul in this class
    Increment numOfCalls
    Assign numOfCalls to CID
int getCID
    Return CID
double getPosx
        Return xcoordinate
double getPosy
        Return ycoordinate

int getUlevel
    Return ul
int getNumOfCalls
    Return numOfCalls
void setPosx (pass double: xcoordinate)
    Assign xcoordinate to xcoordinate in this class
void setPosy (pass double: ycoordinate)
    Assign ycoordinate to ycoordinate in this class
void setUlevel (pass integer: ul)
    Assign ul to ul in this class
void print
    Print (start from the left and set width to 10) CID, (start from the left and set width to 13) ul, (start from the left) "(", (start from the left) xcoordinate, (start from the left) ",", (start from the left) ycoordinate, (start from the left and set width to 10) ")", newline
Assign 0 to the static integer variable numOfCalls in class Call

Declare class PositionStation
<u>Create private members:</u>
Declare PoliceCar**: policecars
Declare Call**: calls
Declare integer: ID, numOfCars, numOfCalls
Declare double: xcoordinate, y coordinate
Declare static integer: numOfStations
<u>Create public members:</u>
PoliceStation()
    Assign 0 to numOfCars
    Assign 0 to numOfCalls
    Assign 0 to xcoordinate
    Assign 0 to ycoordinate
    Increment numOfStations
    Assign numOfStations to ID
Call (pass double: xcoordinate, ycoordinate)
    Assign 0 to numOfCars
    Assign 0 to numOfCalls
    Assign xcoordinate to xcoordinate in this class
    Assign ycoordinate to ycoordinate in this class
    Increment numOfStations
    Assign numOfStations to ID
double getPosx
        Return xcoordinate

```
double getPosy
        Return ycoordinate
int getNumOfCars
    Return numOfCars
int getNumOfCalls
    Return numOfCalls
int getID
    Return ID
int getNumOfStations
    Return numOfStations
void setPosx (pass double: xcoordinate)
      Assign xcoordinate to xcoordinate in this class
void setPosy (pass double: ycoordinate)
      Assign ycoordinate to ycoordinate in this class
double * Calcdist (pass: Call*: C1)
        Create a pointer 'distance' to an array of doubles with the size of numOfCars
        Declare integer i and initialize to 0, while i<numOfCars, repeat the following, increment i
                Use the 'i'th element of array 'policecars' to call function 'getPosx', assign the
                returned value to double x1
                Use the 'i'th element of array 'policecars' to call function 'getPosy', assign the
                returned value to double y1
                Use C1 to call function 'getPosx', assign the returned value to double x2
                Use C1 to call function 'getPosy', assign the returned value to double y2
                Assign sqrt((pow(x1-x2),2))+(pow(y1-y2,2))) to the 'i'th element of array
                'distance'
        Return distance
void update
      Declare integer i and initialize to 0, while i<numOfCars, repeat the following, increment i
              Use the 'i'th element of array 'policecars' to call function 'move'
void assign (pass Call*: call)
      Use C1 to call function 'Calcdist', assign the returned value to pointer 'dist'
      Declare integer min and ctr, both initialize to 0
      Create a pointer 'urglev' to an array of integers with the size of numOfCars
      Declare integer i and initialize to 0, while i<numOfCars, repeat the following, increment i
              if the value returned by the 'i'th element of array 'policecars' calling function
                'getLevel' ≤ the value returned by the C1 calling function 'getUlevel'
                Assign i to the 'ctr'th element of array 'urglev'
                Increment ctr
      if ctr=0
         Print "Sorry, there is no police car available for this call, please dial later!", newline
```

Return
Declare integer i and initialize to 0, while i<ctr, repeat the following, increment i
Assign the ‘i’th element of array ‘urglev’ to an integer index
Assign the 0th element of array ‘urglev’ to min
if the ‘index’th element of array ‘dist’ < the ‘min’th element of array ‘dist’
Assign index to min
Print newline
Print “Accident Information:”, newline
Print “The accident with ID ”, Use the C1 to call function ‘getCID’, “ occurred at ”, “(”, Use C1 to call function ‘getPosx’, “,”, Use C1 to call function ‘getPosy, “), with an urgent level of ”, Use C1 to call function ‘getUlevel’, “.”, newline
Print “The police car assigned to this accident has an ID ”, Use the ‘min’th element of array ‘policecars’ to call function ‘getID’, “, with a minimum urgent level of ”, Use the ‘min’th element of array ‘policecars’ to call function ‘getLevel’, “.”, newline
Print “The distance between the accident and the assigned police car is ”, ‘min’th element of array ‘dist’, newline
Use the ‘min’th element of array ‘policecars’ to call function ‘setPosx’ (pass the value returned when using the C1 to call function ‘getPosx’)
Use the ‘min’th element of array ‘policecars’ to call function ‘setPosy’ (pass the value returned when using the C1 to call function ‘getPosy’)
Call function ‘DeleteCar’ (pass min)
Deallocate array ‘urglev’
Deallocate array ‘dist’
void print
Print “For the police station with the ID ”, call function ‘getID’, “:”, newline
Print newline
Print “Available Police Car Information:”, newline
Print “Number of available police cars:”, numOfCars, newline
Print (start from the left and set width to 10) “ID”, (start from the left and set width to 13) “Urg Level”, (start from the left and set width to 10) “Location”, newline
Declare integer i and initialize to 0, while i<numOfCars, repeat the following, increment i
Use the ‘i’th element of array ‘policecars’ to call function ‘print’
Print newline
Print “Call Information:”, newline
Print “Number of calls:”, numOfCalls, newline
Print (start from the left and set width to 10) “ID”, (start from the left and set width to 13) “Urg Level”, (start from the left and set width to 10) “Location”, newline
Declare integer i and initialize to 0, while i<numOfCalls, repeat the following, increment i
Use the ‘i’th element of array ‘calls’ to call function ‘print’
Print newline

void AddPolicecar (pass PoliceCar*: policecar)

Create a pointer ‘newpolicecars’ to a pointer to an array with the size of numOfCars+1

Declare integer i and initialize to 0, while i<numOfCars, repeat the following, increment i

Assign the ‘i’th element of array ‘policecars’ to the ‘i’th element of array ‘newpolicecars’

Assign policecar to the ‘numOfCars’th element of array ‘newpolicecars’

Assign newpolicecars to policecars

Increment numOfCars

Call function ‘update’

void AddCall (pass Call*: call)

Create a pointer ‘newcalls’ to a pointer to an array with the size of numOfCalls+1

Declare integer i and initialize to 0, while i<numOfCalls, repeat the following, increment i

Assign the ‘i’th element of array ‘calls’ to the ‘i’th element of array ‘newcalls’

Assign call to the ‘numOfCalls’th element of array ‘newcalls’

Assign newcalls to calls

Increment numOfCalls

Call function ‘update’

Call function assign (pass call)

void DeleteCar (pass integer: index)

Create a pointer ‘newpolicecars’ to a pointer to an array with the size of numOfCars-1

Declare integer i and initialize to 0, while i<index, repeat the following, increment i

Assign the ‘i’th element of array ‘policecars’ to the ‘i’th element of array ‘newpolicecars’

Declare integer i and initialize to index, while i<numOfCars-1, repeat the following, increment i

Assign the ‘i+1’th element of array ‘policecars’ to the ‘i’th element of array ‘newpolicecars’

Assign newpolicecars to policecars

Decrement numOfCars

~PoliceStation

Declare integer i and initialize to 0, while i<numOfCars, repeat the following, increment i

Deallocate the ‘i’th element of array ‘policecars’

Deallocate array ‘policecars’

Declare integer i and initialize to 0, while i<numOfCalls, repeat the following, increment i

Deallocate the ‘i’th element of array ‘calls’

Deallocate array ‘calls’

Assign 0 to the static integer variable numOfStations in class PoliceStation

Main function:

```
Declare integer: numOfCars, ul, choice (initialize to 0)
        double: xcoordinate, ycoordinate
Create a pointer 'PS' to an object with type 'PoliceStation', call function 'PoliceStation'
Print "Please enter the number of police cars you want to add", newline
Read value into numOfCars
Print (start from the left and set width to 10) "ID", (start from the left and set width to 13) "Urg
    Level", (start from the left and set width to 10) "Location", newline
Set the seed for the random number generator to time
Declare integer i and initialize to 0, while i<numOfCars, repeat the following, increment i
        Create a pointer 'P1' with type 'PoliceCar', call function 'PoliceCar' (pass PS)
        Use 'PS' to call function 'AddPolicecar' (pass P1)
        Use 'P1' to call function 'print'
Print newline
Print "Thank you for dialing 999, please provide the location of the accident with the form 'x y',
    and its urgent level", newline
Read value into xcoordinate, ycoordinate, ul accordingly
Repeat the following while xcoordinate>100 OR xcoordinate<-100 OR ycoordinate>100 OR
                        ycoordinate<-100 OR ul<1 OR ul>3
        Print "Invalid inputs, please provide again the location and the urgent level of the
            accident", newline
        Read value into xcoordinate, ycoordinate, ul accordingly
Create a pointer 'C1' with type 'Call', call function 'Call' (pass xcoordinate, ycoordinate, ul)
Use 'PS' to call function 'AddCall' (pass C1)
Print newline
Use 'PS' to call function 'print'
Repeat the following while choice≠3
        Print "Please choose what you want to do next:", newline
        Print "1. Add police car(s)", newline
        Print "2. Add a call", newline
        Print "3. Exit", newline
        Read value into choice
        Repeat the following while choice<1 OR choice>3
            Print "Invalid inputs, please re-enter a number between 1, 2, and 3", newline
            Read value into choice
        If choice=1
          Print newline
          Print "Please enter the number of police cars you want to add", newline
          Read value into numOfCars
          Print (start from the left and set width to 10) "ID", (start from the left and set width to
```

13) “Urg Level”, (start from the left and set width to 10) “Location”, newline
Set the seed for the random number generator to time
Declare integer i and initialize to 0, while i<numOfCars, repeat the following, increment i
Create a pointer ‘P1’ with type ‘PoliceCar’, call function ‘PoliceCar’ (pass PS)
Use ‘PS’ to call function ‘AddPolicecar’ (pass P1)
Use ‘PS’ to call function ‘print’
Continue
If choice=2
Print newline
Print “Thank you for dialing 999, please provide the location of the accident with the form ‘x y’, and its urgent level”, newline
Read value into xcoordinate, ycoordinate, ul accordingly
Repeat the following while xcoordinate>100 OR xcoordinate<-100 OR ycoordinate>100 OR ycoordinate<-100 OR ul<1 OR ul>3
Print “Invalid inputs, please provide again the location and the urgent level of the accident”, newline
Read value into xcoordinate, ycoordinate, ul accordingly
Create a pointer ‘C1’ with type ‘Call’, call function ‘Call’ (pass xcoordinate, ycoordinate, ul)
Use ‘PS’ to call function ‘AddCall’ (pass C1)
Print newline
Use ‘PS’ to call function ‘print’
Continue
If choice=3
Print newline
Print “Thank you for using the program, goodbye!”, newline
Exit with 1

Step 4: Code of implementation

```
/*--------------------------------*/
/*Name: Zichen Jing (Nancy), NetID: zj2256 */
/*        Date: April 24, 2023        */
/*         Assignment 3.cpp         */
/*--------------------------------*/

/*---------------------------------------START----------------------------------------
-----*/

#include <iostream>
#include <iomanip>
#include <cmath>
#include <cstdlib>
#include <ctime>

using namespace std;

class PoliceStation;

class PoliceCar
{
private:
    int ID; //police car ID
    double xcoordinate; //x position of the police car
    double ycoordinate; //y position of the police car
    PoliceStation *policestation; //reference to the class police station
    int mul; //minimum urgent level
    static int numOfCars; //static number of police cars, cannot be changed by the
                          program or the user

public:
    //default constructor
    PoliceCar(PoliceStation* policestation){
        xcoordinate=0;
        ycoordinate=0;
        mul=0;
        numOfCars++; //start with 0, increment to 1 when the first police car is
                     created
        ID=numOfCars; //start with 1
        this->policestation=policestation; //reference to the class police station
    }

    //non-default constructor
    PoliceCar(double xcoordinate, double ycoordinate, int mul, PoliceStation*
              policestation){
        this->xcoordinate=xcoordinate; //user decide
        this->ycoordinate=ycoordinate; //user decide
        this->mul=mul; //user decide
        numOfCars++; //decide internally by the program
        ID=numOfCars; //decide internally by the program
        this->policestation=policestation; //reference to the class police station
    }

    //getters of the data members
    int getID(){return ID;}
    double getPosx(){return xcoordinate;}
    double getPosy(){return ycoordinate;}
    int getLevel(){return mul;}
    int getNumOfCars(){return numOfCars;}
```

```
    //setters of the data members
    void setPosx(double xcoordinate){this->xcoordinate=xcoordinate;}
    void setPosy(double ycoordinate){this->ycoordinate=ycoordinate;}
    void setLevel(int mul){this->mul=mul;}

    //move the police car to a randomly new location within (-100,100)x(-100,100),
    change the urgent level of the police car randomly to a number between 1, 2, and 3
    void move(){
        xcoordinate=rand_pos();
        ycoordinate=rand_pos();
        mul=rand_mul();
    }

    // generate a random number from 0 to 200
    double rand_pos(){
        return double(rand()%201-100);
    }

    //generate a random number from 1 to 3
    double rand_mul(){
        return double(rand()%3+1);
    }

    //display the information (ID, minimum urgent level, 2D position) about the police
    car
    void print(){
     cout<<left<<setw(10)<<ID<<left<<setw(13)<<mul<<left<<"("<<left<<xcoordinate<<
         left<<","<<left<<ycoordinate<<left<<setw(10)<<")"<<endl;
    }
};
int PoliceCar::numOfCars=0; //initialize the number of police cars to 0

class Call
{
private:
    int CID; //ID of the call
    double xcoordinate; //x position of the call
    double ycoordinate; //y position of the call
    int ul; //urgent level
    static int numOfCalls; //static number of calls, cannot be changed by the program
                           or the user

public:
    //default constructor
    Call(){
        xcoordinate=0;
        ycoordinate=0;
        ul=0;
        numOfCalls++; //start with 0, increment to 1 when the first accident calls
        CID=numOfCalls; //start with 1
    }

    //non-default constructor
    Call(double xcoordinate, double ycoordinate, int ul){
        this->xcoordinate=xcoordinate; //user decide
        this->ycoordinate=ycoordinate; //user decide
        this->ul=ul; //user decide
        numOfCalls++; //decide internally by the program
        CID=numOfCalls; //decide internally by the program
    }
```

```
    //getters of the data members
    int getCID(){return CID;}
    double getPosx(){return xcoordinate;}
    double getPosy(){return ycoordinate;}
    int getUlevel(){return ul;}
    int getNumOfCalls(){return numOfCalls;}

    //setters of the data members
    void setPosx(double xcoordinate){this->xcoordinate=xcoordinate;}
    void setPosY(double ycoordinate){this->ycoordinate=ycoordinate;}
    void setUlevel(int ul){this->ul=ul;}

    //display the information (ID, urgent level, 2D position) about the call
    void print(){
     cout<<left<<setw(10)<<CID<<left<<setw(13)<<ul<<left<<"("<<left<<xcoordinate<<
         left<<","<<left<<ycoordinate<<left<<setw(10)<<")"<<endl;
    }
};
int Call::numOfCalls=0; //initialize the number of calls to 0

class PoliceStation
{
private:
    int ID; //police station ID
    PoliceCar ** policecars; //reference to the array of PoliceCar
    int numOfCars; //the number of police cars
    Call ** calls; //reference to the array of Call
    int numOfCalls; //the number of calls
    double xcoordinate; //x position of the police station
    double ycoordinate; //y position of the police station
    static int numOfStations; //static number of police stations, cannot be changed by
                              the program or the user

public:
    //default constructor, all values set internally by the program
    PoliceStation(){
        //number of police cars and calls are both set to zero, location of the police
        station is set to (0,0)
        numOfCars=0;
        numOfCalls=0;
        xcoordinate=0;
        ycoordinate=0;
        numOfStations++; //start with 0, increment to 1 when the first police car is
                         created
        ID=numOfStations; //start with 1
    }

    //non-default constructor
    PoliceStation(double xcoordinate, double ycoordinate){
        //number of police cars and calls are both set to zero, location of the police
        station is determined by the user
        numOfCars=0;
        numOfCalls=0;
        this->xcoordinate=xcoordinate; //user decide
        this->ycoordinate=ycoordinate; //user decide
        numOfStations++;
        ID=numOfStations;
    }

    //getters of the data members
    int getID(){return ID;}
```

```
double getPosx(){return xcoordinate;}
double getPosy(){return ycoordinate;}
int getNumOfCars(){return numOfCars;}
int getNumOfCalls(){return numOfCalls;}
int getNumOfStations(){return numOfStations;}

//setters of the data members
void setPosx(double xcoordinate){this->xcoordinate=xcoordinate;}
void setPosy(double ycoordinate){this->ycoordinate=ycoordinate;}

//calculate the distance from the accident to all police cars
double * Calcdist(Call * C1){
    double * distance=new double [numOfCars]; //create an array "distance" with
                                               the size of the number of cars
    //go through all police cars and store the distance into the array
    for (int i=0; i<numOfCars; i++){
        double x1=policecars[i]->getPosx(); //x position of the police car
        double y1=policecars[i]->getPosy(); //y position of the police car
        double x2=C1->getPosx(); //x position of the accident
        double y2=C1->getPosy(); //y position of the accident
        distance[i]=sqrt((pow(x1-x2,2))+(pow(y1-y2,2))); //calculate distance
                                                          between the police car
                                                          and the accident
    }
    return distance;
}

//update the position and the minimum urgent level of each police car randomly
void update(){
    //go through all police cars
    for(int i=0; i<numOfCars; i++){
        policecars[i]->move(); //call the move function
    }
}

//assign the police car based on the status (distance, urgent level) to the
accident
void assign(Call * C1){
    double * dist=Calcdist(C1); //create an array "dist" which stores the distance
                                from each police car to the call
    int min(0), ctr(0);
    int * urglev=new int[numOfCars]; //create an array "urglev" with the size of
                                     the number of cars
    //check available police cars according to urgent level
    for (int i=0; i<numOfCars; i++){
        //the minimum urgent level of the police car must be smaller or equal to
        the urgent level of the accident
        if (policecars[i]->getLevel()<=C1->getUlevel()){
            urglev[ctr]=i; //urglev stores the index of police cars in the array
                           "policecars" with suitable minimum urgent level
            ctr++; //counter to keep track of how many police cars are available
                   according to the urgent level
        }
    }
    //if there is no police car available due to non-matching of the urgent level
    if (ctr==0){
        cout<<"Sorry, there is no police car available for this call, please dial
            later!"<<endl;
        return; //exit the function here, no execution of the rest
    }
    //among police cars with matching urgent levels, find the police car with the
```

```
    nearest distance
    for (int i=0; i<ctr; i++){
        int index=urglev[i]; //get the index of police cars in the array
                             "policecars" with suitable minimum urgent level
        min=urglev[0]; //start with the first one
        if (dist[index]<dist[min])
            min=index; //min is the index of the police cars with the nearest
                       distance to the call
    }
    //display the information about the call and the assigned police car
    cout<<"\n"<<endl; //empty line to make the statements displayed less crowded
    cout<<"Accident Information:"<<endl;
    //information about the call
    cout<<"The accident with ID "<<C1->getCID()<<" occurred at "<<"("<<
        C1->getPosx()<<","<<C1->getPosy()<<"), with an urgent level of"<<
        C1->getUlevel()<<"."<<endl;
    //information about the assigned car
    cout<<"The police car assigned to this accident has an ID "<<
        policecars[min]->getID()<<", with a minimum urgent level of "<<
        policecars[min]->getLevel()<<"."<<endl;
    cout<<"The distance between the accident and the assigned police car is "<<
        dist[min]<<endl;
    //the assigned police car is moved to the location of the call
    policecars[min]->setPosx(C1->getPosx());
    policecars[min]->setPosy(C1->getPosy());
    DeleteCar(min); //the assigned police car is no longer available for other
                    calls therefore deleted
    delete[]urglev; //deallocate memory
    delete[]dist; //deallocate memory
}

//display the information of available police cars (not assigned) and all calls of
the specified police station
void print(){
    cout<<"For the police station with the ID "<<getID()<<":"<<endl;
    cout<<"\n"<<endl; //empty line to make the statements displayed less crowded
    cout<<"Available Police Car Information:"<<endl;
    cout<<"Number of available police cars:"<<numOfCars<<endl; //number of
                                                              available police cars
    cout<<left<<setw(10)<<"ID"<<left<<setw(13)<<"Urg Level"<<left<<setw(10)<<
        "Location"<<endl; //header of table
    //display the information of each police car
    for(int i=0; i<numOfCars; i++){
        policecars[i]->print();
    }
    cout<<"\n"<<endl; //empty line to make the statements displayed less crowded
    cout<<"Call Information:"<<endl;
    cout<<"Number of calls:"<<numOfCalls<<endl;
    cout<<left<<setw(10)<<"ID"<<left<<setw(13)<<"Urg Level"<<left<<setw(10)<<
        "Location"<<endl; //header of table
    //display the information of each call
    for(int i=0; i<numOfCalls; i++){
        calls[i]->print();
    }
    cout<<"\n"<<endl; //empty line to make the statements displayed less crowded
}

//add a police car
void AddPolicecar(PoliceCar * policecar){
    PoliceCar ** newpolicecars=new PoliceCar * [numOfCars+1]; //the size of the
                                                    array becomes original number+1
```

```
    //copy original policecars to the new array
    for(int i=0; i<numOfCars; i++){
        newpolicecars[i]=policecars[i];
    }
    //add the additional police car
    newpolicecars[numOfCars]=policecar;
    //copy the new array back to the original
    policecars=newpolicecars;
    numOfCars++; //increment the number of police cars
    update(); //update the position and the minimum urgent level of all police
              cars including the newly added one
}

//add a call
void AddCall(Call * call){
    Call ** newcalls=new Call * [numOfCalls+1]; //the size of the array becomes
                                                 original number+1
    //copy original calls to the new array
    for(int i=0; i<numOfCalls; i++){
        newcalls[i]=calls[i];
    }
    //add the additional call
    newcalls[numOfCalls]=call;
    //copy the new array back to the original
    calls=newcalls;
    numOfCalls++; //increment the number of calls
    update(); //update the position and the minimum urgent level of all police
              cars
    assign(call); //find the most suitable police car for the call according to
                  urgent level and distance
}

//delete the police car that has been assigned to calls since it is no longer
available
void DeleteCar(int index){
    PoliceCar ** newpolicecars=new PoliceCar * [numOfCars-1]; //the size of the
                                                    array becomes original number-1
    //copy the original policecars to the new array, skip the assigned police car
    which is located at array[index]
    for(int i=0; i<index; i++){
        newpolicecars[i]=policecars[i];
    }
    for(int i=index; i<numOfCars-1; i++){
        newpolicecars[i]=policecars[i+1];
    }
    //copy the new array back to the original
    policecars=newpolicecars;
    numOfCars--; //decrement the number of police cars
}

//destructor of dynamically allocated memory
~PoliceStation(){
    for (int i=0; i<numOfCars; i++){
        delete policecars[i];
    }
    delete[]policecars;
    for (int i=0; i<numOfCalls; i++){
        delete calls[i];
    }
    delete[]calls;
    }
```

```
};
int PoliceStation::numOfStations=0;


int main(){
    int numOfCars, ul, choice(0);
    double xcoordinate, ycoordinate;
    PoliceStation *PS=new PoliceStation; //create a pointer "PS" to an object in
                                         PoliceStation

    cout<<"Please enter the number of police cars you want to add"<<endl;
    cin>>numOfCars;
    //print the header of the information of the created police cars
    cout<<left<<setw(10)<<"ID"<<left<<setw(13)<<"Urg Level"<<left<<setw(10)<<
        "Location"<<endl;
    srand((unsigned)time(NULL)); //set the seed for the random number generator
    //add police cars according to the user's input
    for(int i=0; i<numOfCars; i++){
        PoliceCar *P1=new PoliceCar(PS); //create a pointer "P1" to an object in
                                         PoliceCar, the created police car belongs to
                                         PoliceStation
        PS->AddPolicecar(P1);
        P1->print(); //print the information of the police car
    }

    cout<<"\n"<<endl; //empty line to make the statements displayed less crowded
    cout<<"Thank you for dialing 999, please provide the location of the accident with
        the form 'x y', and its urgent level"<<endl;
    cin>>xcoordinate>>ycoordinate>>ul;
    //check if the user's inputs about the position and urgent level are valid
    while (xcoordinate>100||xcoordinate<-100||ycoordinate>100||
         ycoordinate<-100||ul<1||ul>3){
        cout<<"Invalid inputs, please provide again the location and the urgent level
            of the accident"<<endl;
        cin>>xcoordinate>>ycoordinate>>ul;
    }
    Call*C1=new Call(xcoordinate, ycoordinate, ul); //create a pointer "C1" to an
                                                    object in Call
    PS->AddCall(C1); //add the call to assign an available police car
    cout<<"\n"<<endl; //empty line to make the statements displayed less crowded
    PS->print(); //print the information of remaining police cars and the call

    do{
        //ask the user about their next step
        cout<<"Please choose what you want to do next:"<<endl;
        cout<<"1. Add police car(s)"<<endl;
        cout<<"2. Add a call"<<endl;
        cout<<"3. Exit"<<endl;
        cin>>choice;
        //check if the user's input is valid, must be one of the numbers provided
        while (choice<1||choice>3){
            cout<<"Invalid input, please re-enter a number between 1, 2, and 3"<<endl;
            cin>>choice;
        }

        switch (choice){

            //if the user chose to add more police cars
            case 1:
            {
                cout<<"\n"<<endl; //empty line to make the statements displayed less
```

```
                                crowded
            cout<<"Please enter the number of police cars you want to add"<<endl;
            cin>>numOfCars;
            //print the header of the information of the created police cars
            cout<<left<<setw(10)<<"ID"<<left<<setw(13)<<"Urg Level"<<left<<
                setw(10)<<"Location"<<endl;
            srand((unsigned)time(NULL)); //set the seed for the random number
                                            generator
            //add police cars according to the user's input
            for(int i=0; i<numOfCars; i++){
                PoliceCar *P1=new PoliceCar(PS); //create a pointer "P1" to an
                                                   object in PoliceCar, the created
                                                 police car belongs to PoliceStation
                PS->AddPolicecar(P1);
            }
            PS->print(); //print the information of the updated police cars with
                         the information of calls till now
            continue;
        }

        //if the user chose to add a call
        case 2:
        {
            cout<<"\n"<<endl; //empty line to make the statements displayed less
                                crowded
            cout<<"Thank you for dialing 999, please provide the location of the
                accident with the form 'x y', and its urgent level"<<endl;
            cin>>xcoordinate>>ycoordinate>>ul;
            //check if the user's inputs about the position and urgent level are
            valid
            while (xcoordinate>100||xcoordinate<-100||ycoordinate>100||
                 ycoordinate<-100||ul<1||ul>3){
                cout<<"Invalid inputs, please provide again the location and the
                    urgent level of the accident"<<endl;
                cin>>xcoordinate>>ycoordinate>>ul;
            }
            Call*C1=new Call(xcoordinate, ycoordinate, ul); //create a pointer
                                                              "C1" to an object in Call
            PS->AddCall(C1); //add the call to assign an available police car
            cout<<"\n"<<endl; //empty line to make the statements displayed less
                                crowded
            PS->print(); //print the information of remaining police cars and all
                         the previous calls
            continue;
        }

        //if the user chose to exit the program
        case 3:
        {
            cout<<"\n"<<endl; //empty line to make the statements displayed less
                                crowded
            cout<<"Thank you for using the program, goodbye!"<<endl;
            exit(1); //exit program will receive a return value of 1
        }
    }

}while (choice!=3);//can only exit the do/while loop by choosing 3)Exit

return 0;
}
```

Step 5: Test and Verification

Test Case 1: Adding police cars

```
Please enter the number of police cars you want to add
5
ID        Urg Level    Location
1         3            (53,17)
2         1            (-18,80)
3         2            (-58,54)
4         1            (10,74)
5         2            (-19,-59)


Thank you for dialing 999, please provide the location of the accident with the form 'x y', and its urgent level
```

(The user's input is to add 5 police cars. Successfully added 5 police cars and information about each car is displayed. Including their IDs, 2D positions, and minimum urgent levels. Successfully directed the user to the next step of inputting information about an accident.)

Test Case 2: Add a call (valid inputs)

```
Thank you for dialing 999, please provide the location of the accident with the form 'x y', and its urgent level
50 50 2


Accident Information:
The accident with ID 1 occurred at (50,50), with an urgent level of 2.
The police car assigned to this accident has an ID 3, with a minimum urgent level of 2.
The distance between the accident and the assigned police car is 24.0208


For the police station with the ID 1:


Available Police Car Information:
Number of available police cars:4
ID        Urg Level    Location
1         3            (-39,-31)
2         2            (-38,85)
4         3            (19,-46)
5         3            (68,-29)


Call Information:
Number of calls:1
ID        Urg Level    Location
1         2            (50,50)


Please choose what you want to do next:
1. Add police car(s)
2. Add a call
3. Exit
```

(The user's input about the accident's 2D position is (50,50) with an urgent level of 2. Successfully assigned a police car with a suitable urgent level and the nearest location to the accident. Information about the accident and the assigned police car is displayed. Information about the remaining police cars and the call is displayed. The menu is displayed to the user.)

Test Case 3: Add a call (invalid inputs)

```
Thank you for dialing 999, please provide the location of the accident with the form 'x y', and its urgent level
101 -100 2
Invalid inputs, please provide again the location and the urgent level of the accident
100 -101 1
Invalid inputs, please provide again the location and the urgent level of the accident
100 -100 4
Invalid inputs, please provide again the location and the urgent level of the accident
```

(Each invalid input for an accident has been tested. With x-coordinate out of range (101), y-coordinate out of range (-101), and urgent level out of range (4). In all situations, an error message has been successfully displayed, followed by prompting the user for another value.)

Menu display

```
Please choose what you want to do next:
1. Add police car(s)
2. Add a call
3. Exit
```

Test Case 4: Invalid input in the menu selection

```
Please choose what you want to do next:
1. Add police car(s)
2. Add a call
3. Exit
4
Invalid input, please re-enter a number between 1, 2, and 3
```

(The user's input is 4, which is invalid, an error message is displayed, and prompted the user for another valid value.)

Test Case 5: Selecting 1) Add police car(s)

```
Please choose what you want to do next:
1. Add police car(s)
2. Add a call
3. Exit
1


Please enter the number of police cars you want to add
3
ID        Urg Level    Location
For the police station with the ID 1:


Available Police Car Information:
Number of available police cars:7
ID        Urg Level    Location
1         2            (-18,51)
2         3            (-17,38)
4         3            (-78,40)
5         3            (-37,-25)
6         2            (89,-65)
7         1            (-92,-95)
8         3            (-66,20)


Call Information:
Number of calls:1
ID        Urg Level    Location
1         2            (50,50)


Please choose what you want to do next:
1. Add police car(s)
2. Add a call
3. Exit
```

(The user's input is 1 to add additional police cars. 3 police cars are successfully added and information about unassigned police cars and newly added police cars are displayed. Information about the previous call is also displayed. The menu is re-displayed to the user.)

Test Case 6: Selecting 2) Add a call

```
Please choose what you want to do next:
1. Add police car(s)
2. Add a call
3. Exit
2


Thank you for dialing 999, please provide the location of the accident with the form 'x y', and its urgent level
0 0 1


Accident Information:
The accident with ID 2 occurred at (0,0), with an urgent level of 1.
The police car assigned to this accident has an ID 5, with a minimum urgent level of 1.
The distance between the accident and the assigned police car is 79.2275


For the police station with the ID 1:


Available Police Car Information:
Number of available police cars:6
ID        Urg Level    Location
1         3            (-53,90)
2         2            (-8,16)
4         2            (32,21)
6         3            (13,70)
7         3            (-67,-75)
8         1            (-63,-95)


Call Information:
Number of calls:2
ID        Urg Level    Location
1         2            (50,50)
2         1            (0,0)


Please choose what you want to do next:
1. Add police car(s)
2. Add a call
3. Exit
```

(The user's input is 2 to add an additional call. The user's input about the accident's 2D position is (0,0) with an urgent level of 1. Successfully assigned a police car with a suitable urgent level and the nearest location to the accident. Information about the accident and the assigned police car is displayed. Information about the remaining police cars and all calls is displayed. The menu is re-displayed to the user.)

Test Case 7: Selecting 3) Exit

```
Please choose what you want to do next:
1. Add police car(s)
2. Add a call
3. Exit
3


Thank you for using the program, goodbye!
Program ended with exit code: 1
```

(The user's input is 3 for exit. Message correctly displayed and the program terminates with 1.)